1.
```
#include <iostream>
#include <vector>
#include <algorithm>
using namespace std;

int main() {
    int n;
    cin >> n;               // Enter 0 for an empty array
    vector<int> a(n);

    for (int i = 0; i < n; ++i) {
        cin >> a[i];
    }

    sort(a.begin(), a.end());

    cout << "[";
    for (int i = 0; i < n; ++i) {
        if (i > 0) cout << ", ";
        cout << a[i];
    }
    cout << "]\n";
    return 0;
}
```

```
4
7 7 7 7
[7, 7, 7, 7]


...Program finished with exit code 0
```

2.
```
#include <iostream>
#include <vector>
using namespace std;

void selectionSort(vector<int>& a) {
    int n = static_cast<int>(a.size());

    for (int i = 0; i < n - 1; ++i) {
        int minIndex = i;

        for (int j = i + 1; j < n; ++j) {
            if (a[j] < a[minIndex]) {
                minIndex = j;
            }
        }

        swap(a[i], a[minIndex]);
    }
```

```
}

int main() {
   int n;
   cin >> n;
   vector<int> a(n);
   for (int i = 0; i < n; ++i) cin >> a[i];

   selectionSort(a);

   for (int i = 0; i < n; ++i) {
      if (i) cout << ' ';
      cout << a[i];
   }
   cout << '\n';
   return 0;
}
```

```
5 2 9 1 5 6
1 2 5 6 9


...Program finished with exit code 0
Press ENTER to exit console.
```

```
3. #include <iostream>
#include <vector>
using namespace std;

void bubbleSort(vector<int>& a) {
   int n = static_cast<int>(a.size());

   for (int i = 0; i < n - 1; ++i) {
      bool swapped = false;

      for (int j = 0; j < n - 1 - i; ++j) {
         if (a[j] > a[j + 1]) {
            swap(a[j], a[j + 1]);
            swapped = true;
         }
      }

      if (!swapped) break;  // No swaps means sorted
   }
}

int main() {
   int n;
   cin >> n;
   vector<int> a(n);
```

```
    for (int i = 0; i < n; ++i) cin >> a[i];

    bubbleSort(a);

    for (int i = 0; i < n; ++i) {
        if (i) cout << ' ';
        cout << a[i];
    }
    cout << '\n';
    return 0;
}
```

```
4
5 6 7 2
2 5 6 7


...Program finished with exit code 0
```

4.
```
#include <iostream>
#include <vector>
using namespace std;

void insertionSort(vector<int>& a) {
    for (int i = 1; i < static_cast<int>(a.size()); ++i) {
        int key = a[i];
        int j = i - 1;

        while (j >= 0 && a[j] > key) {
            a[j + 1] = a[j];
            --j;
        }
        a[j + 1] = key;
    }
}

int main() {
    int n;
    cin >> n;
    vector<int> a(n);
    for (int i = 0; i < n; ++i) cin >> a[i];

    insertionSort(a);

    for (int i = 0; i < n; ++i) {
        if (i) cout << ' ';
        cout << a[i];
    }
    cout << '\n';
    return 0;
```

```
}
5
64 25 12 22 11
11 12 22 25 64


...Program finished with exit code 0
Press ENTER to exit console.
```

5.
```
#include <iostream>
#include <vector>
using namespace std;

int findKthPositive(const vector<int>& arr, int k) {
    int candidate = 1;
    int i = 0;

    while (true) {
        if (i < static_cast<int>(arr.size()) && arr[i] == candidate) {
            ++i;
        } else {
            --k;
            if (k == 0) return candidate;
        }
        ++candidate;
    }
}

int main() {
    int n, k;
    cin >> n >> k;
    vector<int> arr(n);
    for (int i = 0; i < n; ++i) cin >> arr[i];

    cout << findKthPositive(arr, k) << '\n';
    return 0;
}
```

```
6
9
4 5 7 0 9 6
12


...Program finished with exit code 0
Press ENTER to exit console.
```

6. #include <iostream>
#include <vector>
using namespace std;

int findPeakElement(const vector<int>& nums) {
    int left = 0;
    int right = static_cast<int>(nums.size()) - 1;

    while (left < right) {
        int mid = left + (right - left) / 2;

        if (nums[mid] < nums[mid + 1]) {
            left = mid + 1;
        } else {
            right = mid;
        }
    }
    return left;
}

int main() {
    int n;
    cin >> n;
    vector<int> nums(n);
    for (int i = 0; i < n; ++i) cin >> nums[i];

    cout << findPeakElement(nums) << '\n';
    return 0;
}

```
5
52 24 11 21 8
3


...Program finished with exit code 0
Press ENTER to exit console.
```

7. #include <iostream>
#include <string>
using namespace std;

int firstOccurrence(const string& haystack, const string& needle) {
    if (needle.empty()) return 0;

    for (int i = 0; i + static_cast<int>(needle.size())
                <= static_cast<int>(haystack.size()); ++i) {
        int j = 0;
        while (j < static_cast<int>(needle.size()) &&
               haystack[i + j] == needle[j]) {
            ++j;

```
        }
        if (j == static_cast<int>(needle.size())) return i;
    }
    return -1;
}

int main() {
    string haystack, needle;
    cin >> haystack >> needle;
    cout << firstOccurrence(haystack, needle) << '\n';
    return 0;
}
```

```
sadbustad sad
0

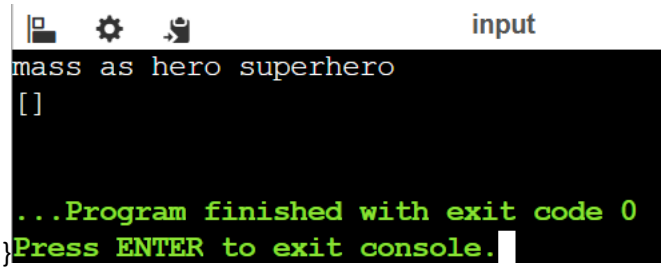

...Program finished with exit code 0
Press ENTER to exit console.
```

8.
```
#include <iostream>
#include <vector>
#include <string>
using namespace std;

int main() {
    int n;
    cin >> n;
    vector<string> words(n);
    for (int i = 0; i < n; ++i) cin >> words[i];

    vector<string> answer;
    for (int i = 0; i < n; ++i) {
        bool found = false;
        for (int j = 0; j < n; ++j) {
            if (i != j && words[j].find(words[i]) != string::npos) {
                found = true;
                break;
            }
        }
        if (found) answer.push_back(words[i]);
    }

    cout << "[";
    for (int i = 0; i < static_cast<int>(answer.size()); ++i) {
        if (i) cout << ", ";
        cout << '"' << answer[i] << '"';
    }
    cout << "]\n";
    return 0;
```



9.
```
#include <iostream>
#include <vector>
#include <cmath>
#include <iomanip>
#include <limits>
using namespace std;

struct Point {
   double x, y;
};

double distanceBetween(const Point& a, const Point& b) {
   double dx = a.x - b.x;
   double dy = a.y - b.y;
   return sqrt(dx * dx + dy * dy);
}

int main() {
   int n;
   cin >> n;
   vector<Point> points(n);
   for (int i = 0; i < n; ++i) cin >> points[i].x >> points[i].y;

   if (n < 2) {
      cout << "At least two points are required.\n";
      return 0;
   }

   double best = numeric_limits<double>::infinity();
   int first = 0, second = 1;

   for (int i = 0; i < n; ++i) {
      for (int j = i + 1; j < n; ++j) {
         double d = distanceBetween(points[i], points[j]);
         if (d < best) {
            best = d;
            first = i;
            second = j;
         }
      }
```

```
    }

    cout << fixed << setprecision(6);
    cout << "Closest pair: (" << points[first].x << ", " << points[first].y
         << ") - (" << points[second].x << ", " << points[second].y << ")\n";
    cout << "Minimum distance: " << best << '\n';
    return 0;
}
```

input

```
4
1 2 4 5 7 8 3 1
Closest pair: (1.000000, 2.000000) - (3.000000, 1.0000
00)
Minimum distance: 2.236068


...Program finished with exit code 0
Press ENTER to exit console.
```

10.
```
#include <iostream>
#include <vector>
#include <algorithm>
#include <cmath>
using namespace std;

struct Point {
    double x, y;
};

double cross(const Point& a, const Point& b, const Point& c) {
    return (b.x - a.x) * (c.y - a.y) -
           (b.y - a.y) * (c.x - a.x);
}

bool samePoint(const Point& a, const Point& b) {
    return fabs(a.x - b.x) < 1e-9 && fabs(a.y - b.y) < 1e-9;
}

vector<Point> convexHullBruteForce(vector<Point> p) {
    sort(p.begin(), p.end(), [](const Point& a, const Point& b) {
        if (a.x != b.x) return a.x < b.x;
        return a.y < b.y;
    });
    p.erase(unique(p.begin(), p.end(), samePoint), p.end());

    int n = static_cast<int>(p.size());
    if (n <= 2) return p;

    vector<bool> boundary(n, false);
    for (int i = 0; i < n; ++i) {
```

```
        for (int j = i + 1; j < n; ++j) {
            bool hasPositive = false, hasNegative = false;

            for (int k = 0; k < n; ++k) {
                double value = cross(p[i], p[j], p[k]);
                if (value > 1e-9) hasPositive = true;
                if (value < -1e-9) hasNegative = true;
            }

            if (!(hasPositive && hasNegative)) {
                for (int k = 0; k < n; ++k) {
                    if (fabs(cross(p[i], p[j], p[k])) < 1e-9) {
                        boundary[k] = true;
                    }
                }
            }
        }
    }

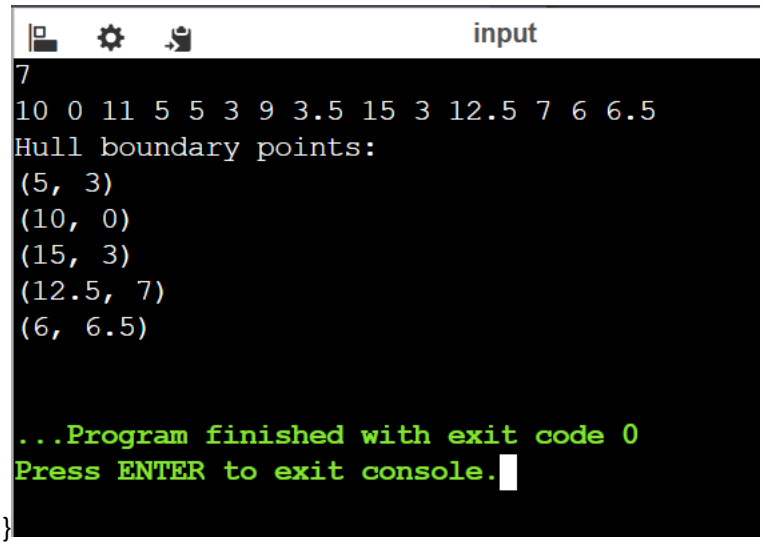
    vector<Point> hull;
    for (int i = 0; i < n; ++i) {
        if (boundary[i]) hull.push_back(p[i]);
    }

    // Order boundary points around their centroid.
    double cx = 0, cy = 0;
    for (const auto& q : hull) { cx += q.x; cy += q.y; }
    cx /= hull.size();
    cy /= hull.size();

    sort(hull.begin(), hull.end(), [cx, cy](const Point& a, const Point& b) {
        return atan2(a.y - cy, a.x - cx) < atan2(b.y - cy, b.x - cx);
    });
    return hull;
}

int main() {
    int n;
    cin >> n;
    vector<Point> points(n);
    for (int i = 0; i < n; ++i) cin >> points[i].x >> points[i].y;

    vector<Point> hull = convexHullBruteForce(points);
    cout << "Hull boundary points:\n";
    for (const auto& p : hull) cout << "(" << p.x << ", " << p.y << ")\n";
    return 0;
```



}

11. #include <iostream>
#include <vector>
#include <algorithm>
#include <cmath>
using namespace std;

struct Point { double x, y; };

double cross(const Point& a, const Point& b, const Point& c) {
  return (b.x-a.x)*(c.y-a.y) - (b.y-a.y)*(c.x-a.x);
}

int main() {
  int n;
  cin >> n;
  vector<Point> p(n);
  for (int i = 0; i < n; ++i) cin >> p[i].x >> p[i].y;

  sort(p.begin(), p.end(), [](const Point& a, const Point& b) {
    if (a.x != b.x) return a.x < b.x;
    return a.y < b.y;
  });
  p.erase(unique(p.begin(), p.end(), [](const Point& a, const Point& b) {
    return fabs(a.x-b.x) < 1e-9 && fabs(a.y-b.y) < 1e-9;
  }), p.end());

  int n2 = static_cast<int>(p.size());
  vector<bool> onHull(n2, false);

  for (int i = 0; i < n2; ++i) {
    for (int j = i + 1; j < n2; ++j) {
      bool pos = false, neg = false;
      for (int k = 0; k < n2; ++k) {

```
            double c = cross(p[i], p[j], p[k]);
            if (c > 1e-9) pos = true;
            if (c < -1e-9) neg = true;
        }
        if (!(pos && neg)) {
            for (int k = 0; k < n2; ++k)
                if (fabs(cross(p[i], p[j], p[k])) < 1e-9)
                    onHull[k] = true;
        }
    }
  }

  vector<Point> hull;
  for (int i = 0; i < n2; ++i) if (onHull[i]) hull.push_back(p[i]);

  double cx = 0, cy = 0;
  for (auto q : hull) { cx += q.x; cy += q.y; }
  cx /= hull.size(); cy /= hull.size();
  sort(hull.begin(), hull.end(), [cx, cy](const Point& a, const Point& b) {
     return atan2(a.y-cy, a.x-cx) < atan2(b.y-cy, b.x-cx);
  });

  cout << "Convex Hull: ";
  for (auto q : hull) cout << "(" << q.x << ", " << q.y << ") ";
  cout << '\n';
  return 0;
```

input

```
4
1 1 4 6 8 1 0 0
Convex Hull: (0, 0) (8, 1) (4, 6)


...Program finished with exit code 0
Press ENTER to exit console.
```

}

12.
```
#include <iostream>
#include <vector>
#include <algorithm>
#include <cmath>
#include <limits>
#include <iomanip>
using namespace std;

struct Point { double x, y; };

double distanceBetween(const Point& a, const Point& b) {
   return hypot(a.x - b.x, a.y - b.y);
}
```

```
int main() {
    int n;
    cin >> n;
    vector<Point> cities(n);
    for (int i = 0; i < n; ++i) cin >> cities[i].x >> cities[i].y;

    if (n < 2) {
        cout << "At least two cities are required.\n";
        return 0;
    }

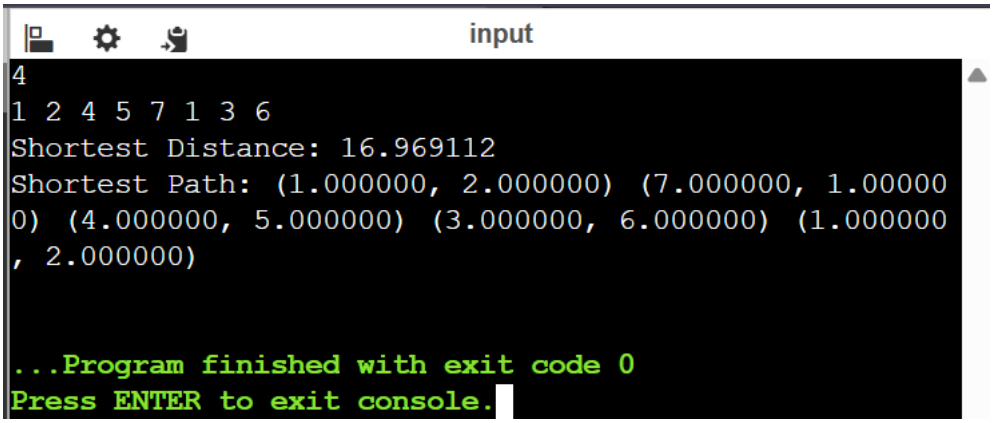
    vector<int> order;
    for (int i = 1; i < n; ++i) order.push_back(i);

    double bestDistance = numeric_limits<double>::infinity();
    vector<int> bestPath;

    do {
        double total = 0.0;
        int previous = 0;
        for (int city : order) {
            total += distanceBetween(cities[previous], cities[city]);
            previous = city;
        }
        total += distanceBetween(cities[previous], cities[0]);

        if (total < bestDistance) {
            bestDistance = total;
            bestPath = {0};
            bestPath.insert(bestPath.end(), order.begin(), order.end());
            bestPath.push_back(0);
        }
    } while (next_permutation(order.begin(), order.end()));

    cout << fixed << setprecision(6);
    cout << "Shortest Distance: " << bestDistance << '\n';
    cout << "Shortest Path: ";
    for (int idx : bestPath)
        cout << "(" << cities[idx].x << ", " << cities[idx].y << ") ";
    cout << '\n';
    return 0;
}
```



13.
```
#include <iostream>
#include <vector>
#include <algorithm>
#include <limits>
using namespace std;

int main() {
    int n;
    cin >> n;
    vector<vector<int>> cost(n, vector<int>(n));
    for (int i = 0; i < n; ++i)
        for (int j = 0; j < n; ++j)
            cin >> cost[i][j];

    vector<int> task(n);
    for (int i = 0; i < n; ++i) task[i] = i;

    int bestCost = numeric_limits<int>::max();
    vector<int> bestAssignment;

    do {
        int total = 0;
        for (int worker = 0; worker < n; ++worker)
            total += cost[worker][task[worker]];

        if (total < bestCost) {
            bestCost = total;
            bestAssignment = task;
        }
    } while (next_permutation(task.begin(), task.end()));

    cout << "Optimal Assignment:\n";
    for (int worker = 0; worker < n; ++worker)
        cout << "Worker " << worker + 1 << " -> Task "
             << bestAssignment[worker] + 1 << '\n';
    cout << "Total Cost: " << bestCost << '\n';
    return 0;
```

}

input

```
3
3 5 7
8 5 12
4 6 9
Optimal Assignment:
Worker 1 -> Task 3
Worker 2 -> Task 2
Worker 3 -> Task 1
Total Cost: 16


...Program finished with exit code 0
Press ENTER to exit console.
```

14.
```
#include <iostream>
#include <vector>
using namespace std;

int main() {
    int n, capacity;
    cin >> n >> capacity;

    vector<int> weights(n), values(n);
    for (int i = 0; i < n; ++i) cin >> weights[i];
    for (int i = 0; i < n; ++i) cin >> values[i];

    int bestValue = 0;
    int bestMask = 0;

    for (int mask = 0; mask < (1 << n); ++mask) {
        int totalWeight = 0;
        int totalValue = 0;

        for (int i = 0; i < n; ++i) {
            if (mask & (1 << i)) {
                totalWeight += weights[i];
                totalValue += values[i];
            }
        }

        if (totalWeight <= capacity && totalValue > bestValue) {
            bestValue = totalValue;
            bestMask = mask;
        }
    }
```

```
    cout << "Optimal Selection: [";
    bool first = true;
    for (int i = 0; i < n; ++i) {
        if (bestMask & (1 << i)) {
            if (!first) cout << ", ";
            cout << i;
            first = false;
        }
    }
    cout << "]\n";
    cout << "Total Value: " << bestValue << '\n';
    return 0;
}
```

